

Almacenamiento en Tarjetas SD

1- Dr. Lic. Roberto Federico FARFÁN

farfan.roberto.f@gmail.com

2- Esp. Ing. Luis Mariano CAMPOS

1,2-Profesor de Periféricos y Comunicaciones en Sistemas Embebidos - **Facultad de Ingeniería – UBA**

1-Profesor de Electrónica, Electrónica Analógica y Digital - **Facultad de Ingeniería - U.N.Sa**

2- **CONICET**

Temas para la semana 3

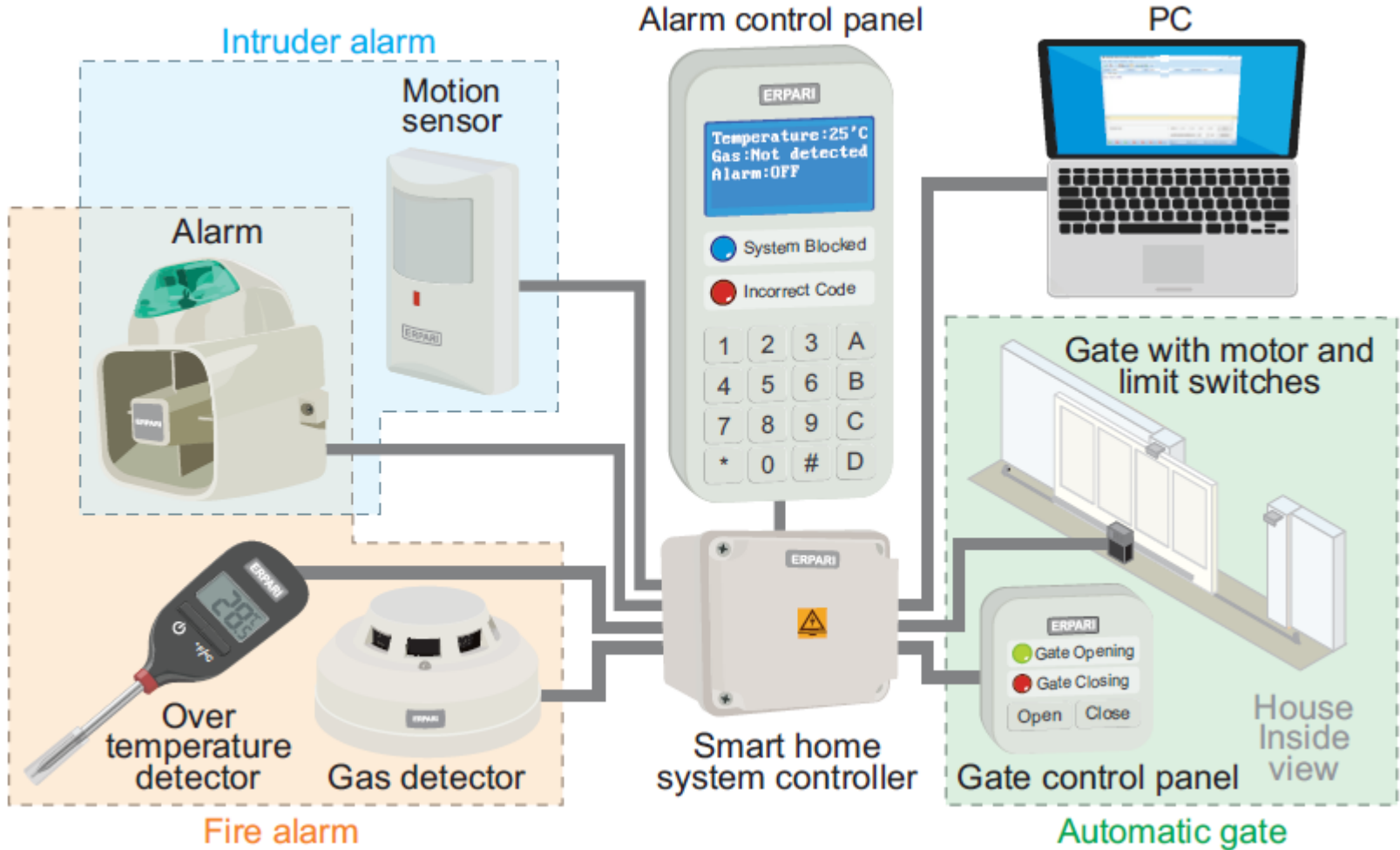
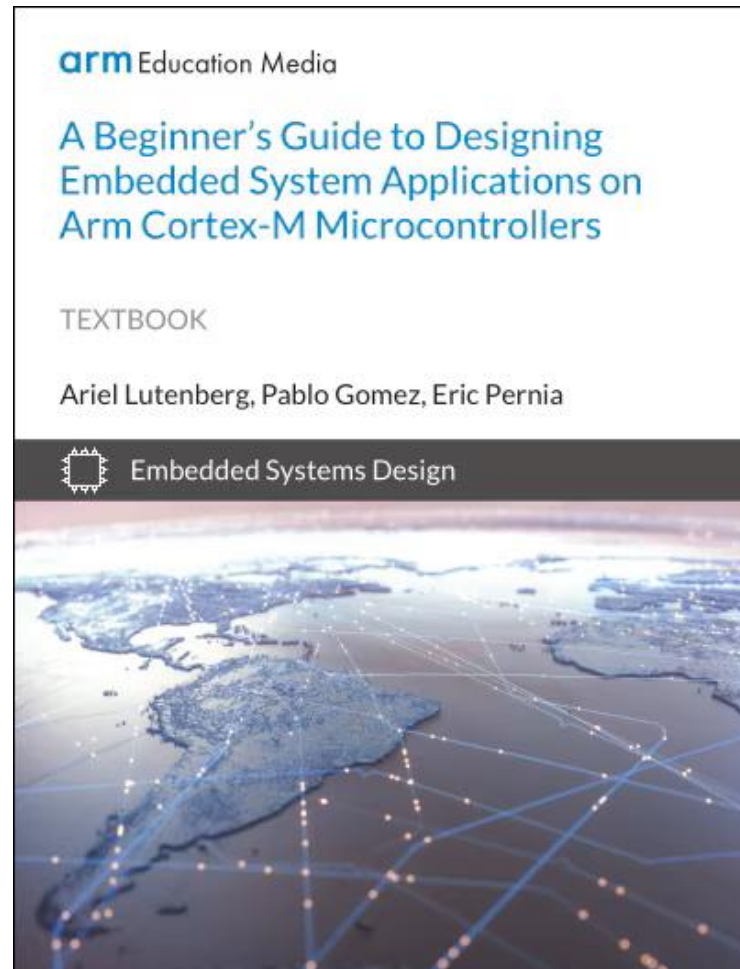


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers.
<https://www.arm.com/resources/education/books/designing-embedded-systems>

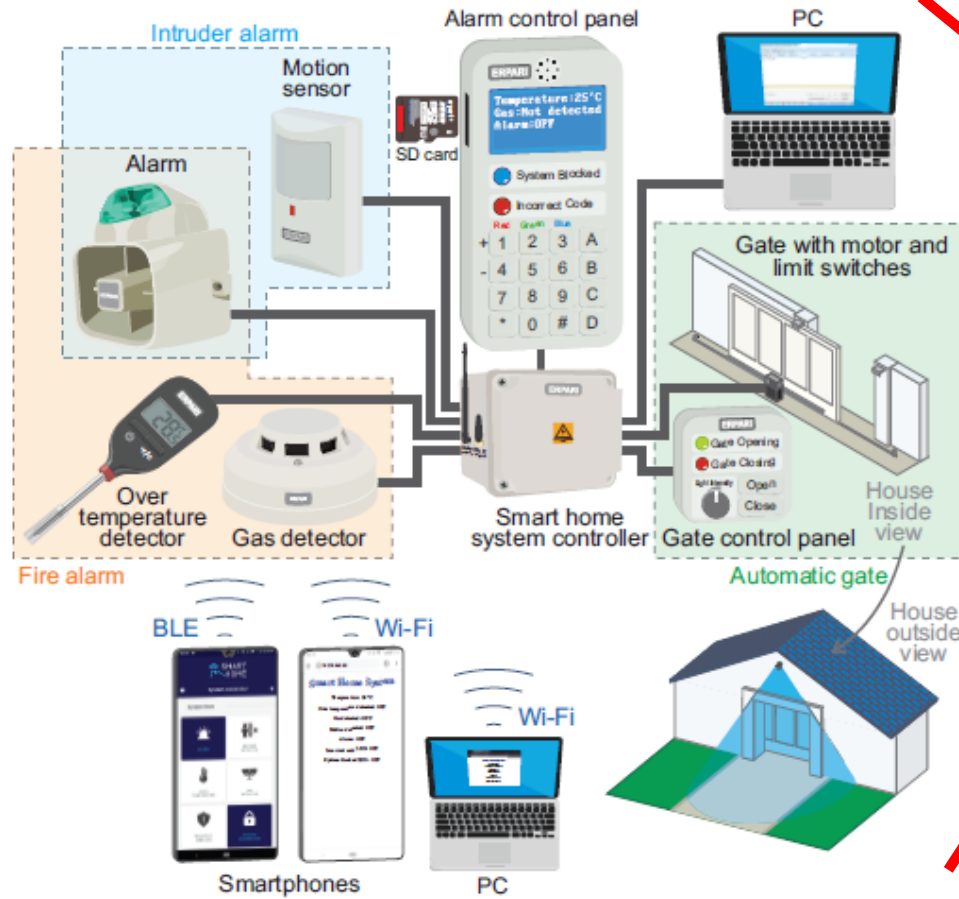
Temas para la semana 3.

3	Contenido de las clases para la semana 3. Teoría: Almacenamiento masivo en tarjetas SD mediante bus SPI. Fundamentos del sistema de archivos FAT32. Práctica: Implementación de un Data Logger para guardar lecturas de sensores con marcas temporales. Desarrollo de un sistema que contenga almacenamiento local y almacenamiento remoto, haciendo uso de una base de datos con Node-RED.	Será informada por el docente a cargo.	1, 2, 3 y 4.
4	Contenido de las clases para la semana 4. Teoría: Modulación por ancho de pulso (PWM). Registros de control de ciclo de trabajo y frecuencia. Sistema de control. Retroalimentación negativa. Sonido. Práctica: Reproducción de mensajes de audio mediante señal PWM. Desarrollo de una interfaz gráfica en Node-RED para la gestión de alarmas.	Será informada por el docente a cargo.	1, 2, 3 y 4.
5	Contenido de las clases para la semana 5. Teoría: Introducción a Bluetooth Low Energy (BLE). Comandos AT y perfiles de comunicación. Práctica: Control para la placa NUCLEO-F446RE desde un smartphone utilizando un módulo BLE. Comunicación entre el microcontrolador STM32F446RE con teléfonos inteligentes y PC para monitoreo y control.	Será informada por el docente a cargo.	1, 2, 3 y 4.

Libro de Referencia



Libro de referencia.



arm Education Media

A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers

TEXTBOOK

Ariel Lutenberg, Pablo Gomez, Eric Pernia

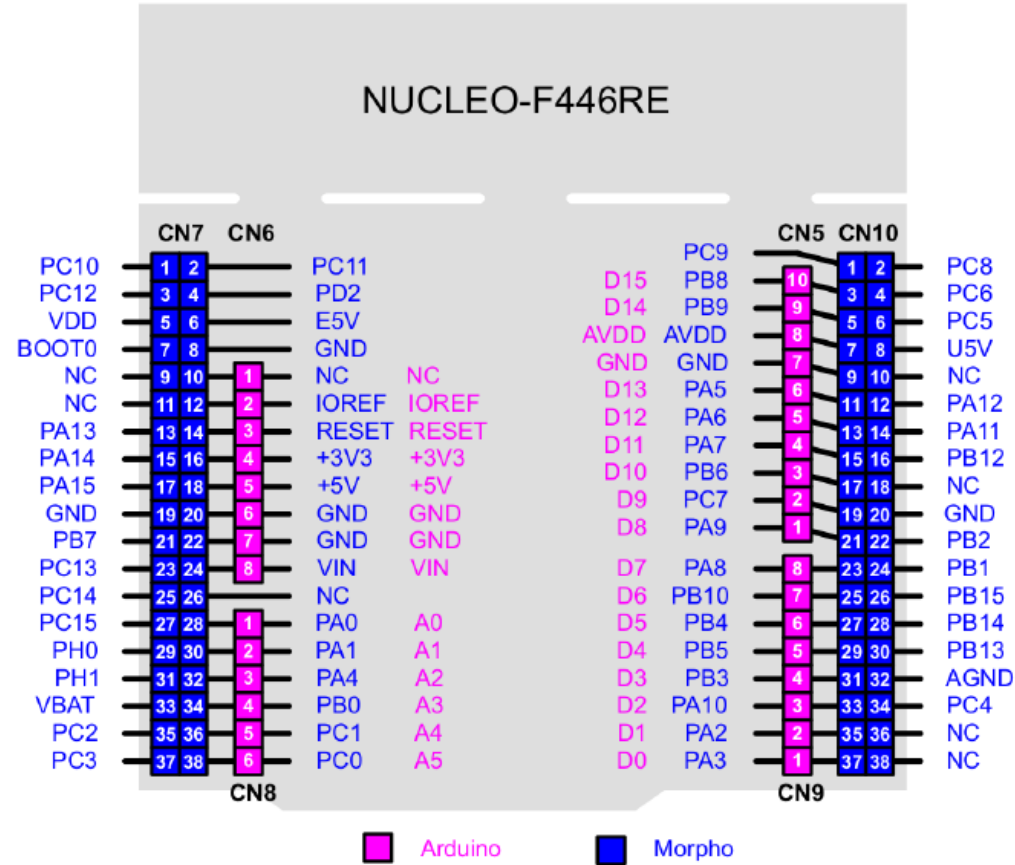


Embedded Systems Design

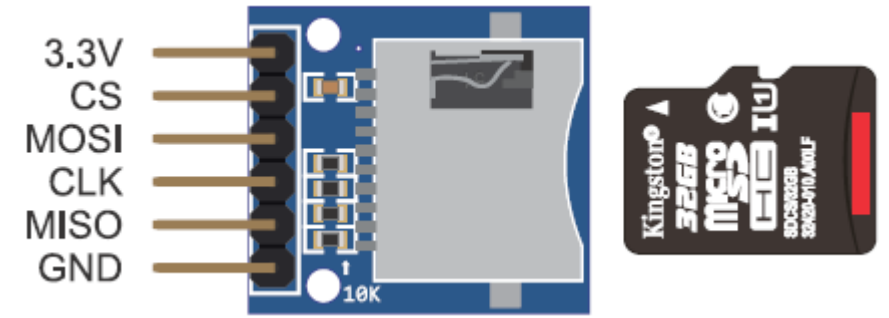


<https://www.arm.com/resources/education/books/designing-embedded-systems>

Repaso: SD.

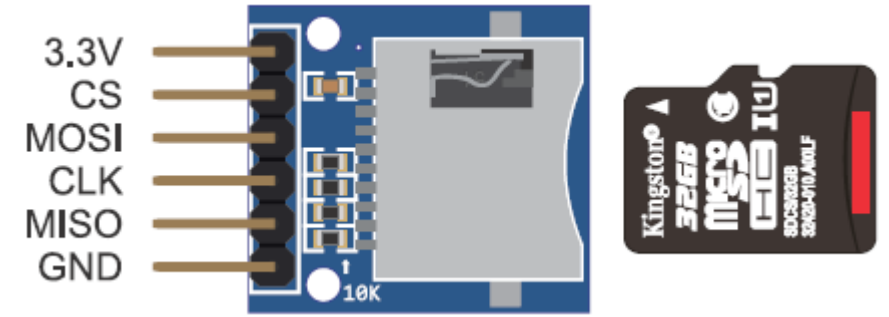


Repaso: SD.



- Conectividad de Hardware:
- Se utiliza una tarjeta micro-SD conectada a la placa NUCLEO mediante el bus SPI.
- Pines utilizados: 3,3V, CS, MOSI, CLK, MISO y GND.
- La tarjeta debe estar formateada como FAT32 para ser compatible.

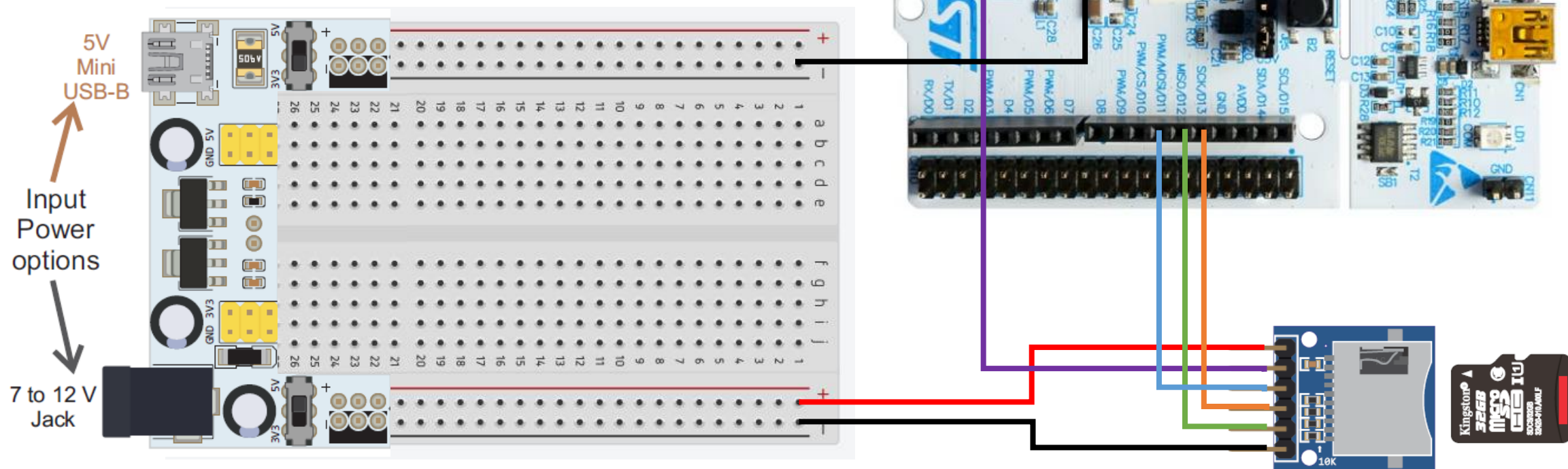
Repaso: SD.



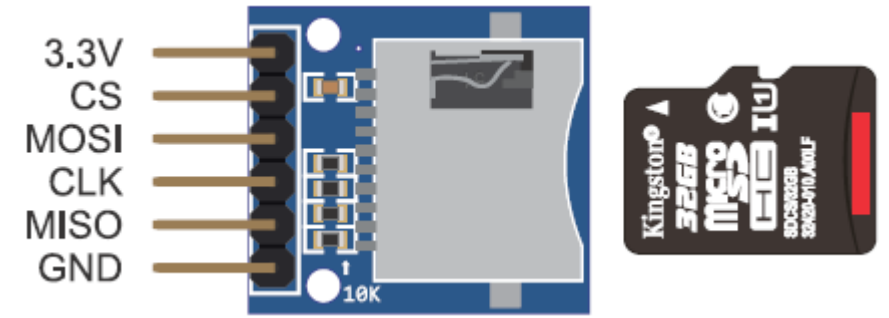
- Conectividad de Hardware:
- Para la conexión de una memoria SD al microcontrolador se emplea el bus de comunicación SPI.
- Características de la Comunicación SPI: bus serie síncrono y full-dúplex.

Repaso: SD.

- Prueba de memoria SD.



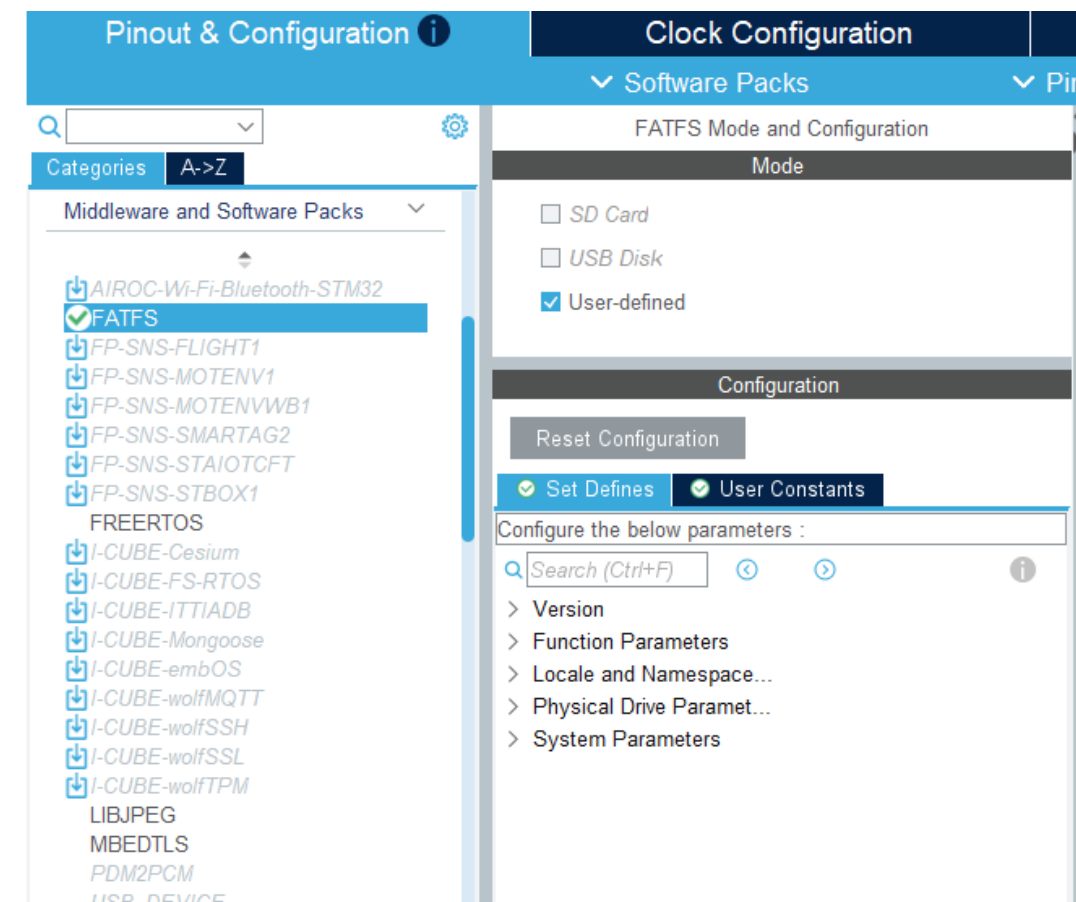
Repaso: SD.



- Arquitectura Manager/Subordinate: El STM32 debe actuar como maestro en la comunicación, generando la señal de reloj (SCLK) y controlando la línea SS (Subordinate Select) para activar el dispositivo deseado.
- Líneas de datos: se utiliza líneas separadas para enviar (MOSI) y recibir (MISO) datos simultáneamente.
- Se suele configurar una frecuencia inicial de 350 kHz para la estabilización y luego se aumenta para la transferencia de datos

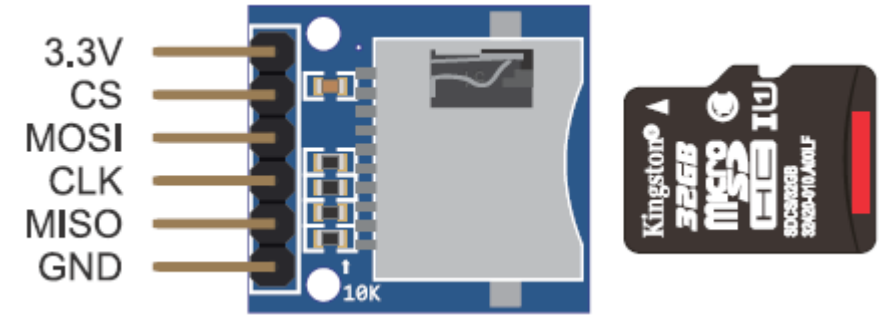
CubeMX:

- HAL (Hardware Abstraction Layer): capa de bajo nivel para controlar periféricos (GPIO, SPI, UART).



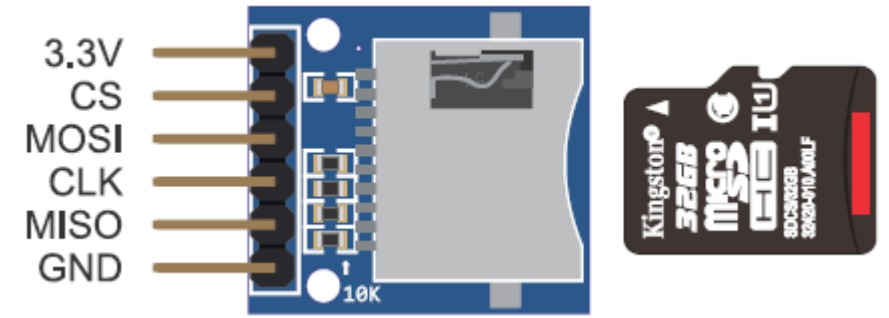
- **Middleware:** servicio de software para tareas complejas, actúa como puente entre la aplicación y el hardware.
- Software Packs: componentes modulares de terceros o del fabricante para extender funcionalidades.

CubeMX:



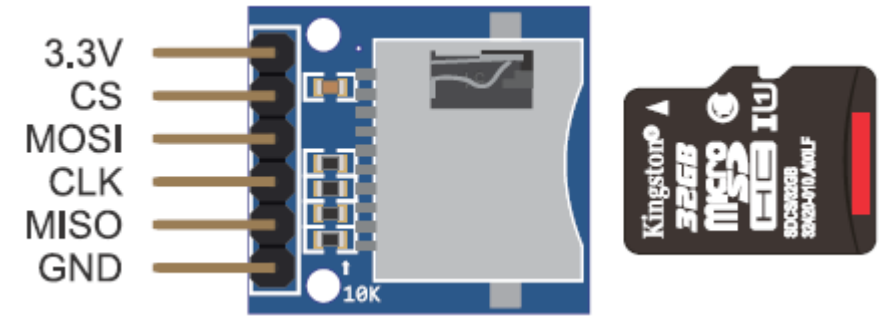
- El conjunto de librerías de la Middleware viven sobre la HAL pero debajo de la lógica de usuario.
- Ejemplo: escribir un sistema de archivos para una pila TCP/IP desde cero podría tomar meses, el IDE cuentan con librerías que nos permiten desarrollar de forma rápida una aplicación.
- Ejemplo: En lugar de mover bit a bit información hacia la memoria SD, le indicamos al middleware: `f_write()`.

CubeMX:



- FatFS: Es un middleware de sistemas de archivos (File System) de código abierto y estándar de la industria.
- Permite que el STM32 gestione archivos y carpetas en medios de almacenamiento (SD) utilizando el formato FAT.
- Portabilidad: **FatFS es independiente del hardware**, necesita una capa intermedia (nuestro desarrollo) para comunicarse con la SD.

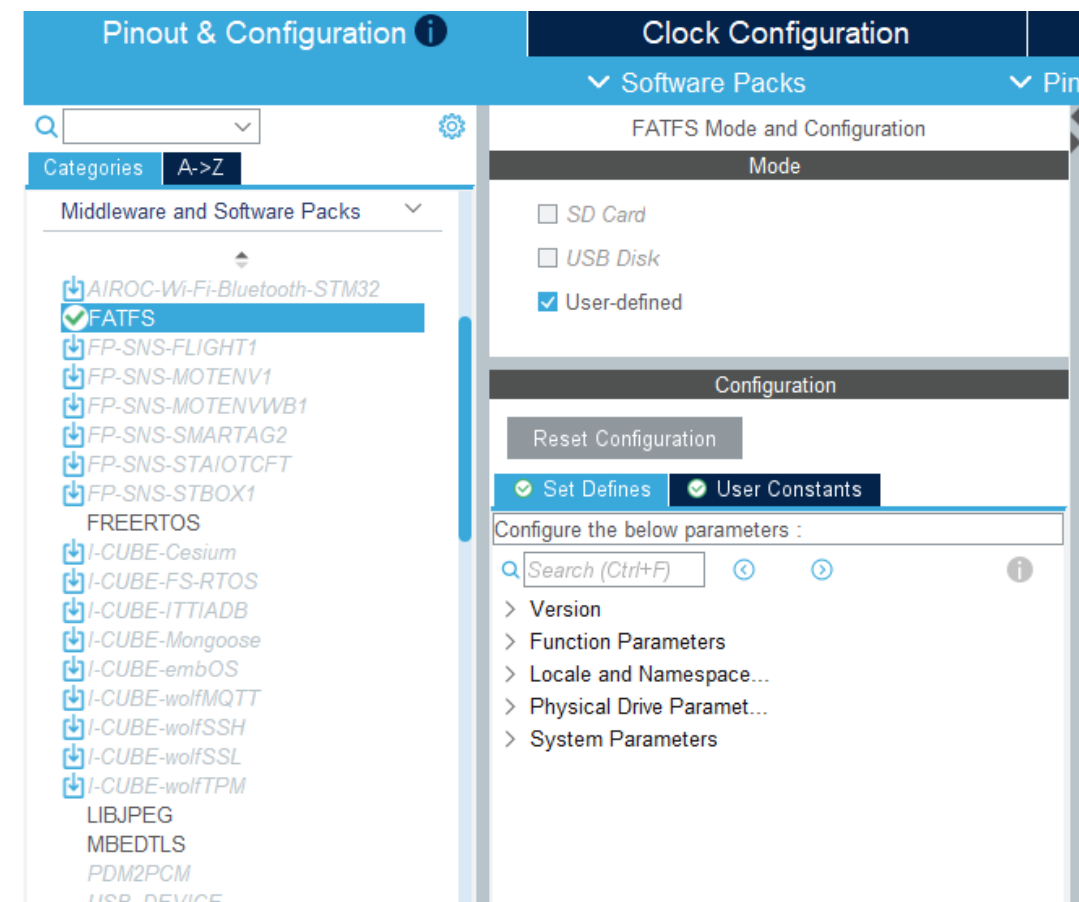
CubeMX:



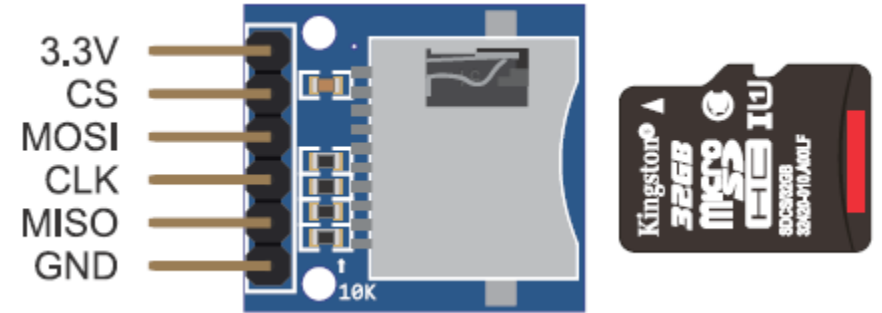
- La arquitectura en capas (El Stack).
- **Nivel superior:** nuestra aplicación (f_open, f_write, f_read).
- **Nivel Medio (Middleware):** FatFS (gestiona la lógica del sistema de archivos).
- **Nivel Inferior (Driver):** gestiona la comunicación SPI (unión de niveles en user_diskio.c).
- **Hardware:** El pin físico de tu STM32 se comunica con la tarjeta SD. 14

CubeMX:

- Archivo "Puente": user_diskio.c.
- FatFS no entiende una comunicación SPI, solo interpreta funciones como disk_read y disk_write.
- El Middleware trabaja como una "caja negra":
- Si el sistema falla, no es por FatFS.

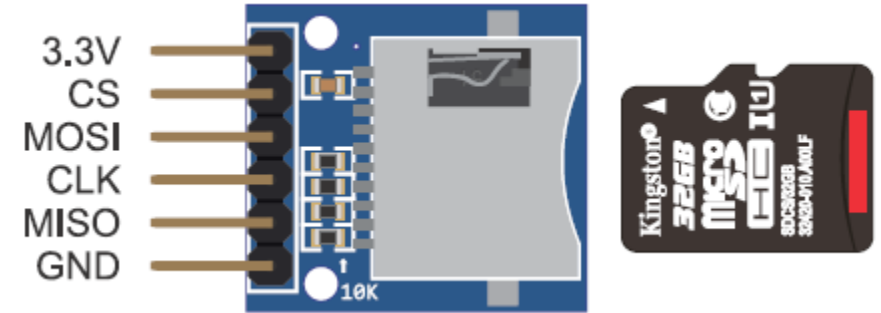


Repaso: SD.



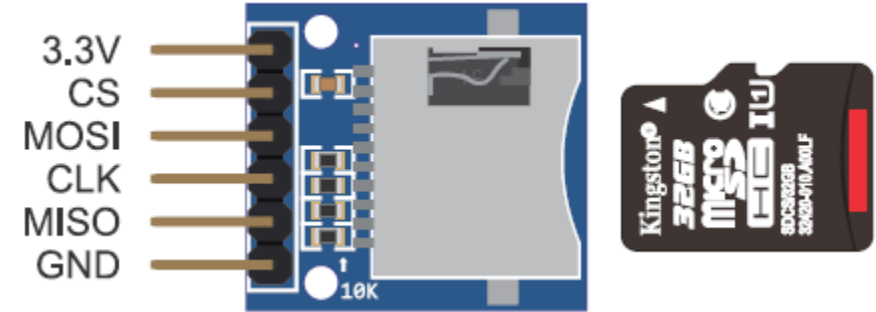
- Resumen:
- El Middleware permite escalar proyectos complejos.
- FatFS es el estándar para memorias SD: es robusto pero requiere que el hardware esté perfectamente sincronizado.
- Nosotros conectamos el hardware real con el software abstracto.

Archivos utilizados:



- **Funciones en fatfs_sd.c: Capa de Comunicación Física.**
- **SELECT / DESELECT:** Controlan el pin CS (Chip Select). Bajo (RESET) activa la tarjeta SD en el bus SPI, alto la desconecta.
- **SPI_TxByte / SPI_TxBuffer:** Envían datos (1 byte o un bloque de bytes) hacia la SD usando el periférico SPI de tu micro.
- **SPI_RxByte / SPI_RxBytePtr:** Reciben datos.

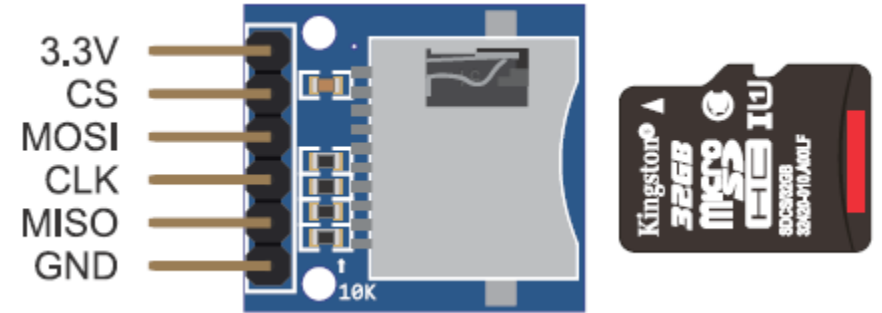
Archivos utilizados:



- Funciones en fatfs_sd.c: Capa de Comunicación Física.

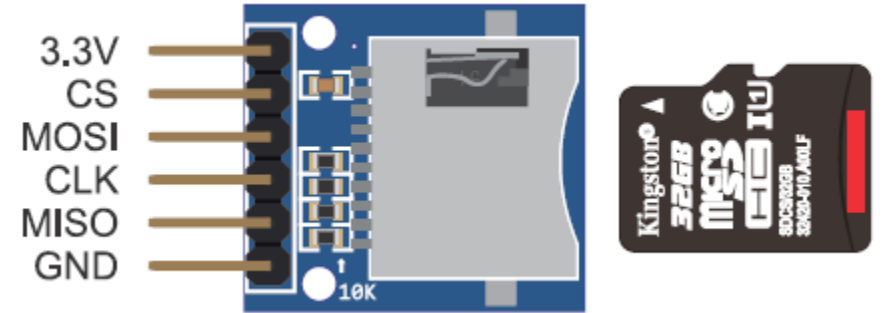
```
/* slave select */
static void SELECT(void) {
    HAL_GPIO_WritePin(SD_CS_PORT, SD_CS_PIN, GPIO_PIN_RESET);
}
/* slave deselect */
static void DESELECT(void) { HAL_GPIO_WritePin(SD_CS_PORT, SD_CS_PIN,
GPIO_PIN_SET);
}
```

Archivos utilizados:



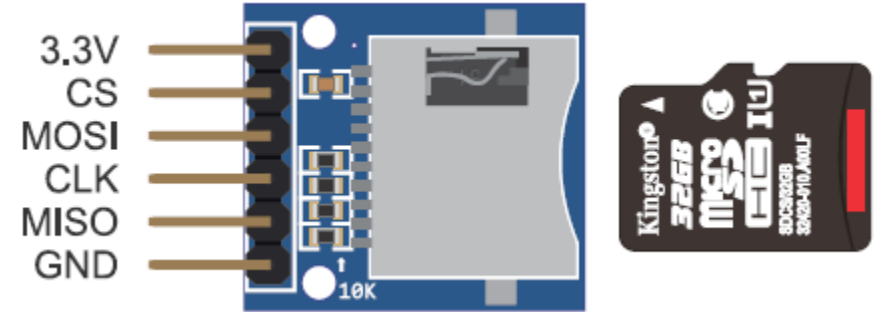
- Archivo **fatfs_sd.c**: Capa de Control de SD (Protocolo de Comandos).
- Funciones gestionan el estado y la lógica interna de la SD.
- **SD_ReadyWait**: Espera a que la tarjeta termine una tarea interna. La tarjeta SD envía 0xFF constantemente cuando está libre.
- **SD_PowerOn / SD_PowerOff**: Inician la secuencia de energía y el comando de "Reset" (CMD0) para poner la tarjeta en modo SPI.

Archivos utilizados:



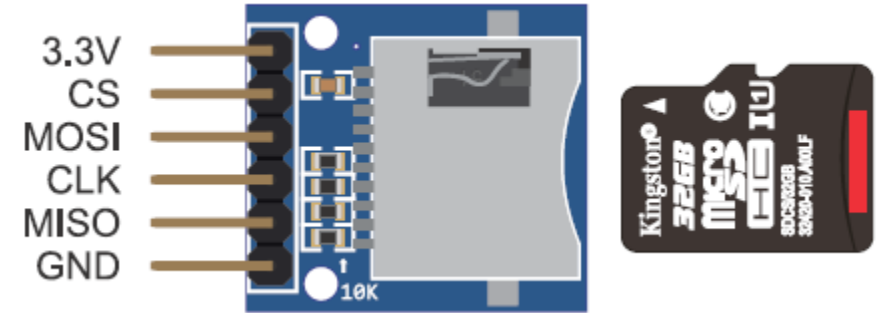
- **Archivo fatfs_sd.c:** Capa de Control de SD (Protocolo de Comandos).
- Estas funciones gestionan el estado y la lógica interna de la SD.
- **SD_SendCmd:** Envía comandos de protocolo SD. Gestiona el número de comando, el argumento, el CRC y espera la respuesta de la tarjeta.
- **SD_RxDataBlock / SD_TxDataBlock:** Manejan la transferencia de un bloque de datos (normalmente 512 bytes).

Archivos utilizados:



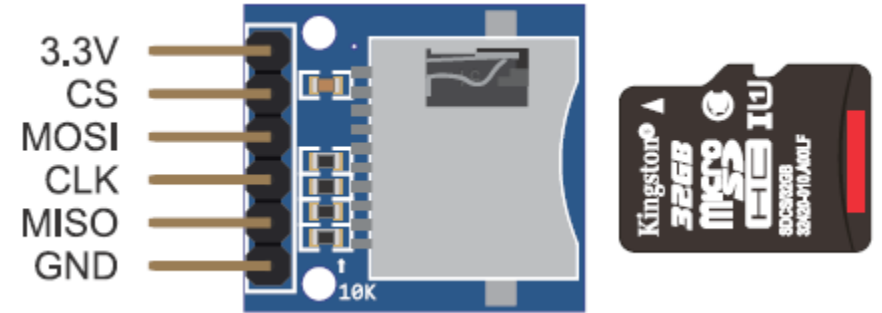
- **Archivo fatfs_sd.c:** Interfaz para FatFS (user_diskio.c)
- **SD_disk_initialize:** La primera función que se llama, si detecta la tarjeta, ajusta la configuración de bloques y pone la tarjeta lista para trabajar.
- **SD_disk_status:** Devuelve el estado actual de la tarjeta (si está inicializada, si tiene errores o si hay disco insertado).

Repaso: SD.



- Archivo `fatfs_sd.c`: Interfaz para FatFS (`user_diskio.c`)
- **SD_disk_initialize**: La primera función que se llama, si detecta la tarjeta, ajusta la configuración de bloques y pone la tarjeta lista para trabajar.
- **SD_disk_status**: Devuelve el estado actual de la tarjeta (si está inicializada, si tiene errores o si hay disco insertado).

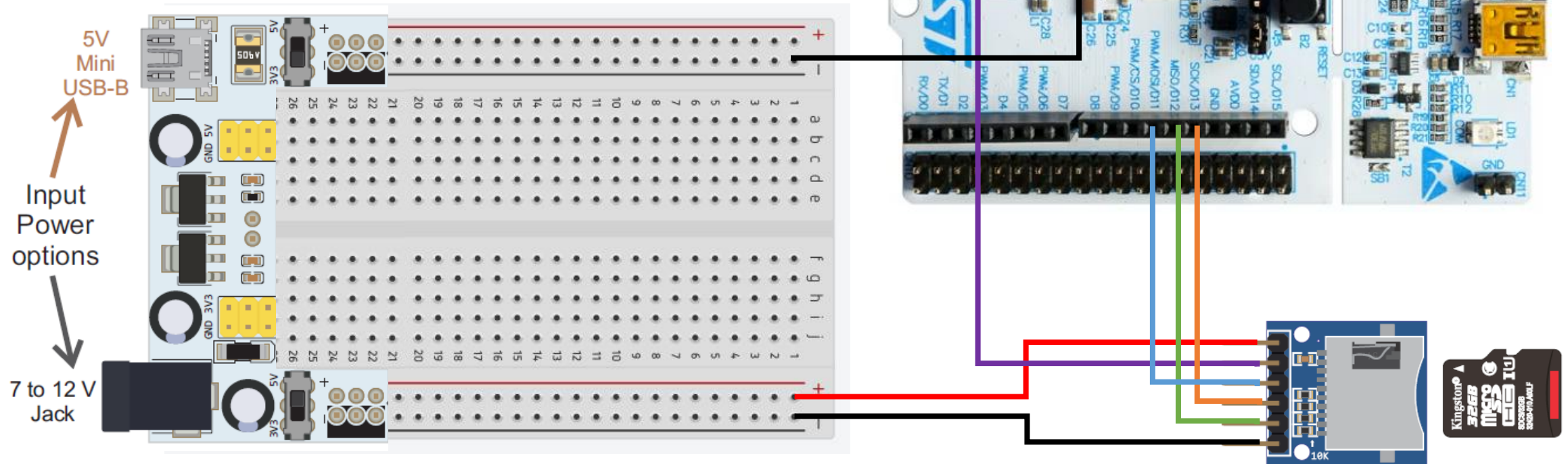
Repaso: SD.



- Archivo `fatfs_sd.c`: Interfaz para FatFS (`user_diskio.c`)
- **SD_disk_read**: función que FatFS llama cuando se quiere leer un archivo.
- **SD_disk_write**: función que FatFS llama para guardar datos.

Prueba de Módulos.

- Prueba de memoria SD.



Problema.

- Función para montar y desmontar una SD.

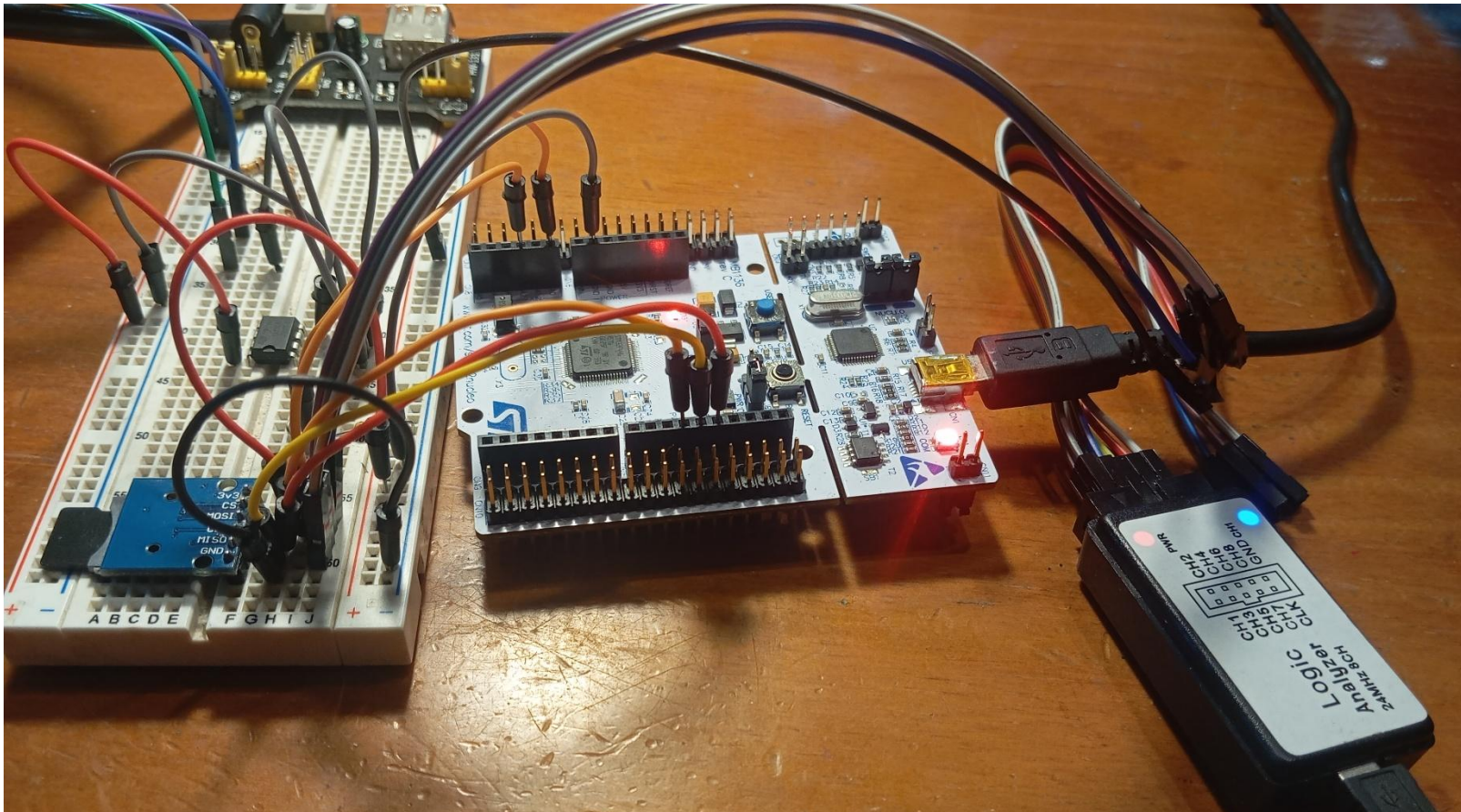
```
void SD_Card_Test(void) {
printf("Start FATFS System\n");
FATFS FatFs;
FIL Fil;
FRESULT FR_Status;
FATFS *FS_Ptr;
UINT RWC, WWC;
DWORD FreeClusters;
uint32_t TotalSize, FreeSpace;
char RW_Buffer[200];

FR_Status = f_mount(&FatFs, "", 1);

if (FR_Status != FR_OK) {
printf("[ERROR] SD Card mount failed! Code = %d\r\n", FR_Status);
return; }
printf("[INFO] SD Card mounted successfully\r\n\n");
HAL_Delay(500);
FR_Status = f_mount(NULL, "", 0);
if (FR_Status != FR_OK) printf("[ERROR] SD Card unmount failed! Code = %d\r\n", FR_Status);
else printf("[INFO] SD Card unmounted safely\r\n\n"); }
```

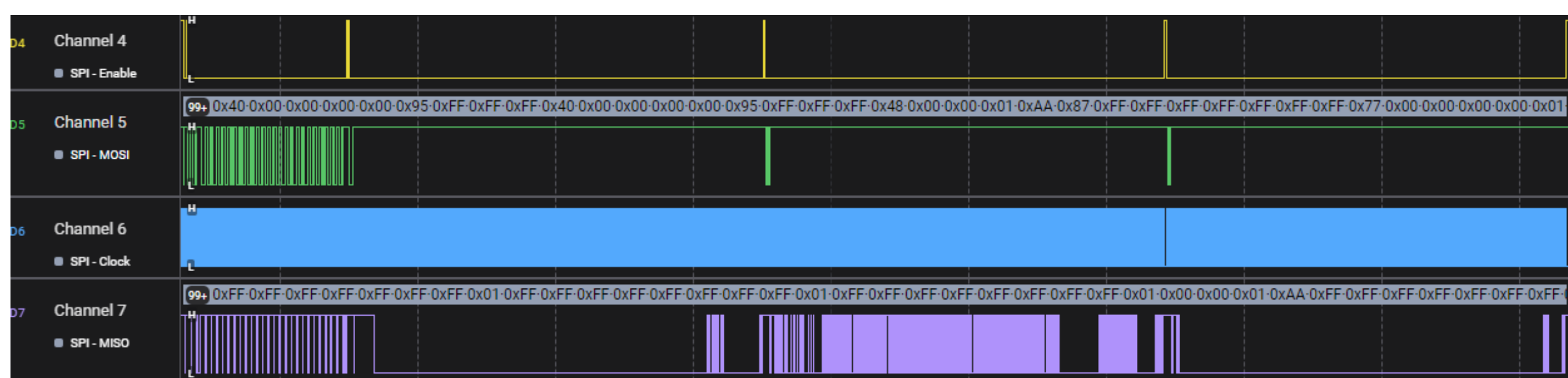
Problema.

- Analizador Lógico.



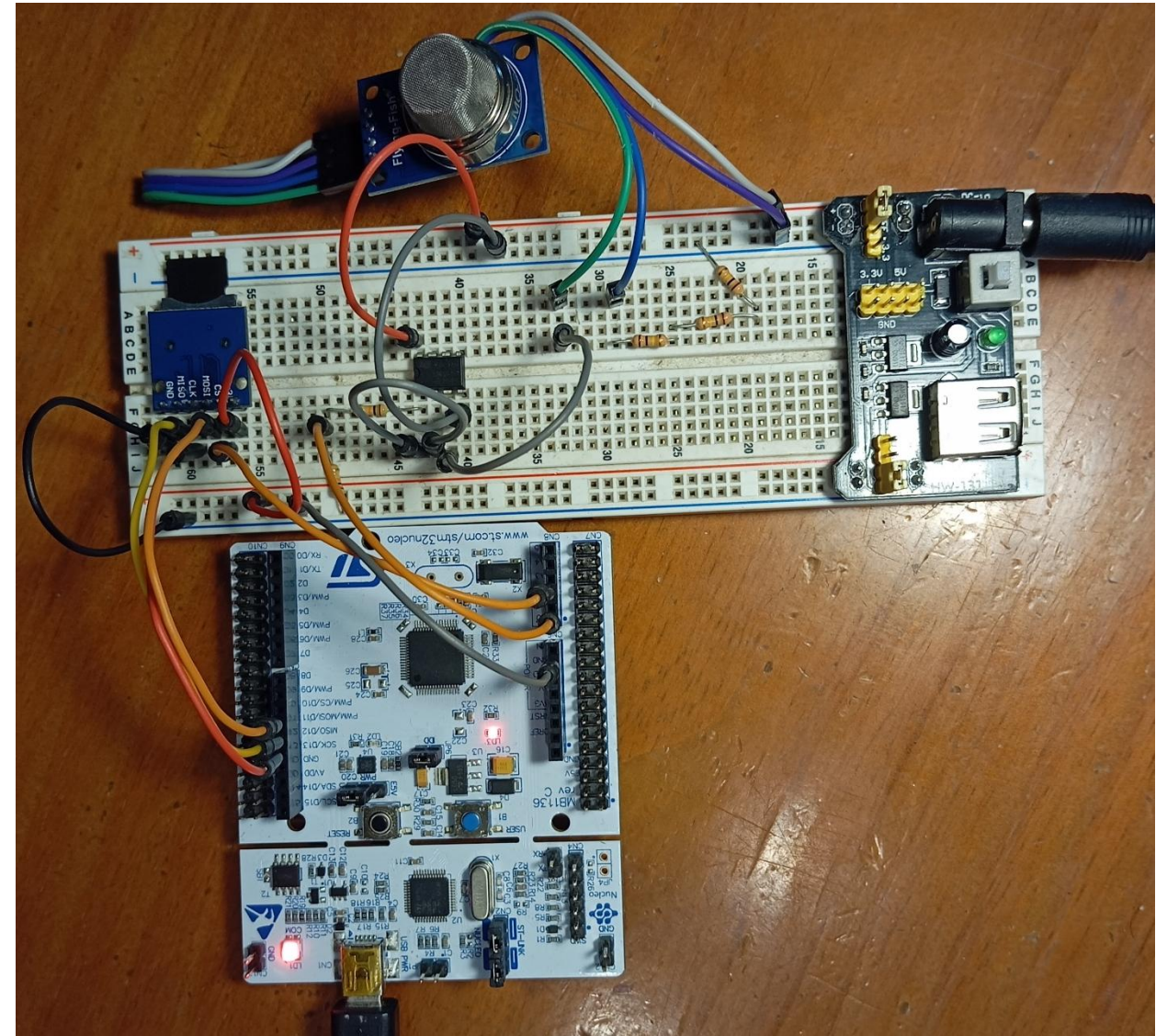
Problema.

- Señal de comunicación en el puerto SPI entre el micro y la SD.



Problemas.

- Incorpore al circuito implementado, el sensor MQ2 y guarde información de las ppm medidas en la SD.



Bibliografía.

Lutenberg A., Gomez P. and Pernia E., 2022. A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Arm Education Media.

Ortiz J.P., Malagón P. J., Cárdenas R. and Gómez J.J., 2025. Fundamentos teóricos de sistemas basados en microcontrolador STM32, Sistemas Digitales II Sistemas Electrónicos, Universidad Politécnica de Madrid.

Cem Ünsalan C., Yücel M. E. and Gürhan H. D., 2018. Digital Signal Processing using Arm Cortex-M based Microcontrollers-Theory and Practice, Arm Education Media.

YIU J., 2019. System-on-Chip Design with Arm® Cortex® -M Processors, Arm Education Media